

NeuroTrace

U-Net Brain Tumor Segmentation — CS 571

Predicts pixel-level tumor masks from LGG brain MRI images using a U-Net trained with combined BCE + Dice loss.

Run cells top to bottom. Make sure the Colab runtime is set to **GPU** (Runtime → Change runtime type → T4 GPU).

1. Install dependencies

```
!pip install -q albumentations opencv-python-headless tqdm scikit-learn
```

2. Upload / mount the dataset

Download **LGG Brain MRI Segmentation** from Kaggle:

<https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation>

Then either:

- **Option A** — Mount Google Drive and point `DATA_DIR` at the extracted folder, or
- **Option B** — Upload the zip directly and unzip it here.

```
import os, glob
from google.colab import drive

drive.mount('/content/drive')

ARCHIVE = '/content/drive/MyDrive/NeuroTrace/archive.zip'
EXTRACT_T0 = '/content/dataset/'
os.makedirs(EXTRACT_T0, exist_ok=True)

!apt-get install -q p7zip-full
!7z x "$ARCHIVE" -o"$EXTRACT_T0" -y

# Find kaggle_3m wherever it landed
matches = glob.glob(f'{EXTRACT_T0}*/kaggle_3m', recursive=True)
if not matches:
    contents = os.listdir(EXTRACT_T0)
    raise FileNotFoundError(f'kaggle_3m not found. Contents: {contents}')

DATA_DIR = matches[0]
print(f'DATA_DIR = {DATA_DIR}')
```

```
Mounted at /content/drive
Reading package lists...
Building dependency tree...
Reading state information...
p7zip-full is already the newest version (16.02+dfsg-8).
0 upgraded, 0 newly installed, 0 to remove and 42 not upgraded.

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,2 CPUs Intel(R) Xeon(R) CPU @ 2.00GHz (50653),A

Scanning the drive for archives:
1 file, 748584920 bytes (714 MiB)

Extracting archive: /content/drive/MyDrive/NeuroTrace/archive.zip
--
Path = /content/drive/MyDrive/NeuroTrace/archive.zip
Type = zip
Physical Size = 748584920

Everything is Ok

Files: 15720
Size:      2112621008
Compressed: 748584920
DATA_DIR = /content/dataset/kaggle_3m
```

3. Clone / upload the project files

If you pushed this repo to GitHub, clone it. Otherwise upload `data.py`, `model.py`, `train.py`, and `eval.py` manually.

```
# Clone from GitHub (replace with your repo URL)
# !git clone https://github.com/<your-username>/NeuroTrace.git
# %cd NeuroTrace

# Or just verify the files are present in the current directory
import os
for f in ['data.py', 'model.py', 'train.py', 'eval.py']:
    status = '✓' if os.path.exists(f) else '✗ MISSING'
    print(f"{status} {f}")
```

```
✓ data.py
✓ model.py
✓ train.py
✓ eval.py
```

4. Imports & device

```
import torch
from data import build_dataloaders
from model import UNet, count_parameters
from train import train
from eval import evaluate, visualize_predictions, plot_training_history

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
```

```
Using device: cuda
```

5. Hyperparameters

```
IMAGE_SIZE = 256
BATCH_SIZE = 32
EPOCHS = 60
LR = 1e-4
WEIGHT_DECAY = 1e-5
CHECKPOINT = 'best_model.pth'
```

6. Build dataloaders

```
train_loader, val_loader, test_loader = build_dataloaders(
    DATA_DIR,
    image_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
)
```

```
Dataset split - train: 2749 val: 590 test: 590
/usr/local/lib/python3.12/dist-packages/albumentations/core/validation.py:114: UserWarning: ShiftScaleRotate is a spe
original_init(self, **validated_kwargs)
```

7. Visualize a sample batch

```
import matplotlib.pyplot as plt
import numpy as np

images, masks = next(iter(train_loader))

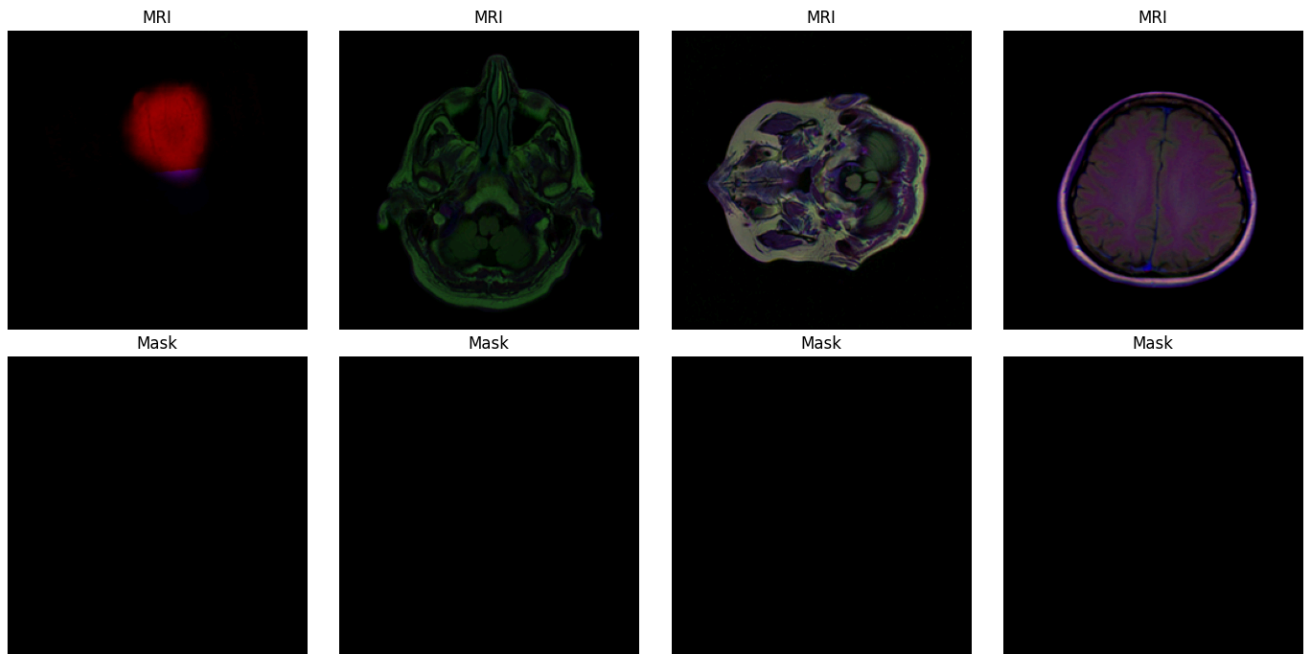
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])

fig, axes = plt.subplots(2, 4, figsize=(14, 7))
for i in range(4):
    img = images[i].numpy().transpose(1, 2, 0)
```

```

img = np.clip(std * img + mean, 0, 1)
axes[0, i].imshow(img)
axes[0, i].set_title('MRI')
axes[0, i].axis('off')
axes[1, i].imshow(masks[i].squeeze(), cmap='gray')
axes[1, i].set_title('Mask')
axes[1, i].axis('off')
plt.tight_layout()
plt.show()

```



8. Build the U-Net model

```

model = UNet(in_channels=3, out_channels=1)
count_parameters(model)

```

```

Total parameters:    31,037,633
Trainable parameters: 31,037,633
31037633

```

9. Train

```

history = train(
    model,
    train_loader,
    val_loader,
    epochs=EPOCHS,
    lr=LR,
    weight_decay=WEIGHT_DECAY,
    checkpoint_path=CHECKPOINT,
    device=DEVICE,
    use_amp=True,
)

```

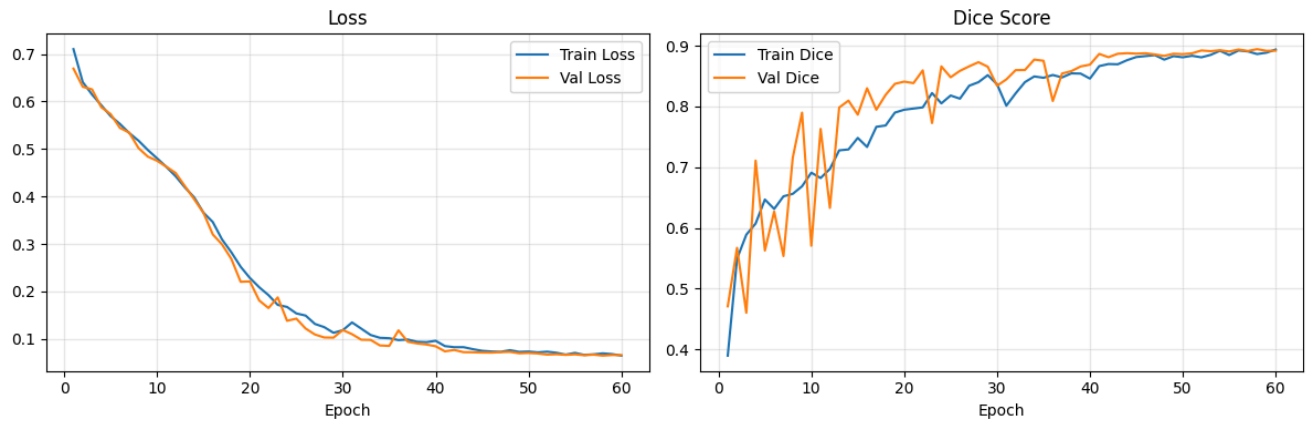
```

✓ Saved best model (val_dice=0.8408)
Epoch 021/60 train_loss: 0.2088 train_dice: 0.7964 val_loss: 0.1809 val_dice: 0.8383
Epoch 022/60 train_loss: 0.1922 train_dice: 0.7984 val_loss: 0.1650 val_dice: 0.8593
✓ Saved best model (val_dice=0.8593)
Epoch 023/60 train_loss: 0.1715 train_dice: 0.8220 val_loss: 0.1874 val_dice: 0.7726
Epoch 024/60 train_loss: 0.1673 train_dice: 0.8050 val_loss: 0.1381 val_dice: 0.8658
✓ Saved best model (val_dice=0.8658)
Epoch 025/60 train_loss: 0.1536 train_dice: 0.8179 val_loss: 0.1426 val_dice: 0.8480
Epoch 026/60 train_loss: 0.1491 train_dice: 0.8127 val_loss: 0.1218 val_dice: 0.8585
Epoch 027/60 train_loss: 0.1315 train_dice: 0.8341 val_loss: 0.1092 val_dice: 0.8660
✓ Saved best model (val_dice=0.8660)
Epoch 028/60 train_loss: 0.1248 train_dice: 0.8399 val_loss: 0.1030 val_dice: 0.8728
✓ Saved best model (val_dice=0.8728)
Epoch 029/60 train_loss: 0.1128 train_dice: 0.8515 val_loss: 0.1026 val_dice: 0.8653
Epoch 030/60 train_loss: 0.1182 train_dice: 0.8364 val_loss: 0.1187 val_dice: 0.8341
Epoch 031/60 train_loss: 0.1346 train_dice: 0.8011 val_loss: 0.1098 val_dice: 0.8445
Epoch 032/60 train_loss: 0.1214 train_dice: 0.8215 val_loss: 0.0982 val_dice: 0.8597
Epoch 033/60 train_loss: 0.1082 train_dice: 0.8400 val_loss: 0.0978 val_dice: 0.8600
Epoch 034/60 train_loss: 0.1022 train_dice: 0.8493 val_loss: 0.0861 val_dice: 0.8771
✓ Saved best model (val_dice=0.8771)
Epoch 035/60 train_loss: 0.1014 train_dice: 0.8472 val_loss: 0.0852 val_dice: 0.8752
Epoch 036/60 train_loss: 0.0974 train_dice: 0.8517 val_loss: 0.1179 val_dice: 0.8089
Epoch 037/60 train_loss: 0.0987 train_dice: 0.8477 val_loss: 0.0940 val_dice: 0.8542
Epoch 038/60 train_loss: 0.0939 train_dice: 0.8544 val_loss: 0.0903 val_dice: 0.8581
Epoch 039/60 train_loss: 0.0931 train_dice: 0.8540 val_loss: 0.0882 val_dice: 0.8658
Epoch 040/60 train_loss: 0.0960 train_dice: 0.8458 val_loss: 0.0844 val_dice: 0.8688
Epoch 041/60 train_loss: 0.0848 train_dice: 0.8663 val_loss: 0.0737 val_dice: 0.8864
✓ Saved best model (val_dice=0.8864)
Epoch 042/60 train_loss: 0.0825 train_dice: 0.8698 val_loss: 0.0766 val_dice: 0.8810
Epoch 043/60 train_loss: 0.0826 train_dice: 0.8693 val_loss: 0.0720 val_dice: 0.8867
✓ Saved best model (val_dice=0.8867)
Epoch 044/60 train_loss: 0.0784 train_dice: 0.8761 val_loss: 0.0717 val_dice: 0.8876
✓ Saved best model (val_dice=0.8876)
Epoch 045/60 train_loss: 0.0749 train_dice: 0.8813 val_loss: 0.0712 val_dice: 0.8869
Epoch 046/60 train_loss: 0.0735 train_dice: 0.8829 val_loss: 0.0711 val_dice: 0.8875
Epoch 047/60 train_loss: 0.0727 train_dice: 0.8844 val_loss: 0.0722 val_dice: 0.8856
Epoch 048/60 train_loss: 0.0762 train_dice: 0.8770 val_loss: 0.0726 val_dice: 0.8832
Epoch 049/60 train_loss: 0.0727 train_dice: 0.8827 val_loss: 0.0696 val_dice: 0.8867
Epoch 050/60 train_loss: 0.0734 train_dice: 0.8809 val_loss: 0.0704 val_dice: 0.8860
Epoch 051/60 train_loss: 0.0716 train_dice: 0.8834 val_loss: 0.0692 val_dice: 0.8874
Epoch 052/60 train_loss: 0.0731 train_dice: 0.8808 val_loss: 0.0669 val_dice: 0.8921
✓ Saved best model (val_dice=0.8921)
Epoch 053/60 train_loss: 0.0708 train_dice: 0.8848 val_loss: 0.0676 val_dice: 0.8910
Epoch 054/60 train_loss: 0.0668 train_dice: 0.8916 val_loss: 0.0663 val_dice: 0.8925
✓ Saved best model (val_dice=0.8925)
Epoch 055/60 train_loss: 0.0705 train_dice: 0.8845 val_loss: 0.0673 val_dice: 0.8905
Epoch 056/60 train_loss: 0.0662 train_dice: 0.8921 val_loss: 0.0656 val_dice: 0.8937
✓ Saved best model (val_dice=0.8937)
Epoch 057/60 train_loss: 0.0674 train_dice: 0.8906 val_loss: 0.0668 val_dice: 0.8913
Epoch 058/60 train_loss: 0.0695 train_dice: 0.8862 val_loss: 0.0647 val_dice: 0.8943
✓ Saved best model (val_dice=0.8943)
Epoch 059/60 train_loss: 0.0679 train_dice: 0.8884 val_loss: 0.0660 val_dice: 0.8916
Epoch 060/60 train_loss: 0.0648 train_dice: 0.8937 val_loss:
-----
Training complete. Best val Dice: 0.8943

```

✓ 10. Plot training curves

```
plot_training_history(history)
```



11. Evaluate on the test set

```
# Load the best checkpoint before evaluating
model.load_state_dict(torch.load(CHECKPOINT, map_location=DEVICE))
```

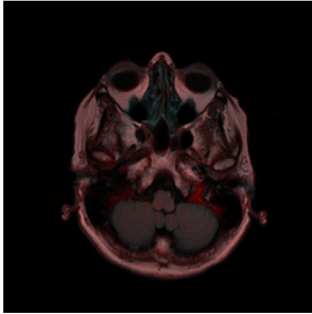
```
results = evaluate(model, test_loader, DEVICE)
```

```
Evaluating: 100%|██████████| 19/19 [00:10<00:00, 1.79it/s]Test Dice: 0.8783 | Test IoU: 0.7866
```

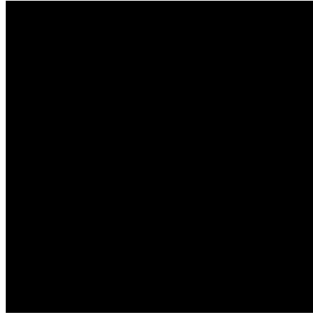
12. Visualize predictions

```
visualize_predictions(model, test_loader, DEVICE, n_samples=4)
```

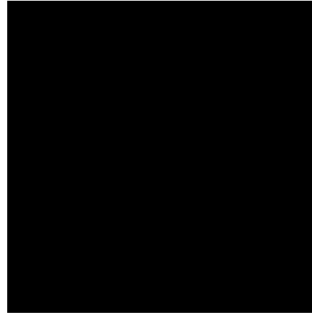
MRI Image



Ground Truth



Prediction



Overlay

